

i. Code Logic and Implementation (R1 – R4)

The OtoDecks application was developed using C++ and the JUCE framework to provide a fully functional DJ interface.

- **R1 (Core Functionality):** The basic playback features—such as loading audio files, playing two tracks simultaneously, mixing volumes, and adjusting playback speeds—are implemented via the `DJAudioPlayer` and `DeckGUI` classes. The `MainComponent` mixes the outputs of both decks using a `MixerAudioSource`.
- **R2 (Music Library):** A custom `PlaylistComponent` manages the user's music library. It utilizes a `TableListBox` to display track titles and durations. The application allows users to load multiple files via a `FileChooser` and assigns them to either deck using dynamically generated routing buttons.
- **R3 (Hot Cues):** The `DeckGUI` component features an array of 8 `TextButton` objects acting as Hot Cues. These buttons capture the track's current relative position (`player->getPositionRelative()`) when clicked and store it in an array.
- **R4 (Loop Sampler):** A standalone `SamplerComponent` captures audio buffers from either the microphone or the decks. This component displays saved `.wav` files in a `ListBox` and uses an `AudioTransportSource` to preview them.

Code Fragment: Hot Cue Logic (R3)

The following code snippet demonstrates how the application assigns, jumps to, and clears individual Hot Cues:

```
// R3A & R3B: Hot Cue interaction

for (int i = 0; i < 8; ++i)
{
    if (button == &cueButtons[i])
    {
        if (ModifierKeys::getCurrentModifiers().isShiftDown())
        {
            // Shift+Click: Clear individual cue (R3B)

            cuePositions[i] = -1.0;

            cueButtons[i].setColour(TextButton::buttonColourId, Colours::darkgrey);
        }
        else if (cuePositions[i] == -1.0)
        {
            // Regular Click on empty: Assign cue (R3A)

            cuePositions[i] = player->getPositionRelative();

            cueButtons[i].setColour(TextButton::buttonColourId, Colours::orange);
        }
        else
        {
            // Regular Click on filled: Jump to cue (R3A)

            player->setPositionRelative(cuePositions[i]);
        }

        saveCues(); // Save state changes
    }
}
```

Discussion: This loop identifies which of the 8 cue buttons was clicked. It checks if the **Shift** key is held to clear the cue (setting the value to `-1.0`). If the cue is empty, it assigns the current track position; if it already contains a valid timestamp, it jumps the player to that position.

ii. Assumptions, Decisions, and Unique Features

Several design decisions were made to ensure stability and meet persistence requirements:

- **Default BPM Assumption (R5D):** Because implementing real-time Fast Fourier Transform (FFT) transient-detection is highly complex, the application assumes a default track tempo of 120.0 BPM (`double baseBPM{120.0};`).
- **File Persistence Locations (R2C, R3D, R4C):** The application relies on local files to save states between sessions. The `PlaylistComponent` writes paths to a `playlistData.txt` file, while the `DeckGUI` saves track-specific cues to a `.cues` file using standard `std::ofstream` operations.
- **Sampler Desktop Directory (R4C):** To avoid potential macOS sandboxing and write-permission crashes, the `SamplerComponent` explicitly saves its recorded `.wav` files to a dedicated folder on the user's Desktop (`File::userDesktopDirectory`) rather than the application's working directory.

iii. Beats Per Minute (BPM) Investigation (R5)

What is BPM and How is it Used? (R5A)

BPM (Beats Per Minute) is the primary metric used by DJs to match track tempos.

- **Scenario 1 (Beatmatching):** A DJ uses the BPM value to manually match the tempos of two tracks using the speed slider so that their drum hits align, allowing for a seamless musical crossfade.
- **Scenario 2 (Time-Synced Effects):** DJs use BPM data to lock audio effects, such as echo or delay, to the track's tempo. This ensures the effect repeats on exact musical intervals rather than sounding disjointed.

Alternative Techniques (R5B)

Modern DJ applications offer automated alternatives to manual BPM mixing:

- **The "Sync" Button:** Software automatically calculates and locks the BPM and phase of multiple tracks together instantly.
- **Phase Meters / Visual Beatgrids:** These allow DJs to visually align transients on screen instead of relying solely on numeric BPM values.

Conceptual BPM Calculation (R5C)

Programmatically calculating BPM requires Transient Detection.

- The software analyzes the audio signal's energy over time, filtering for low-frequency spikes like kick drums.
- By detecting these sharp amplitude peaks (transients) and measuring the time intervals between them, the algorithm calculates how many beats occur within a 60-second window to yield the BPM value.

Application Implementation (R5D)

In this application, the BPM is visualized dynamically. A dedicated `Label` on the `DeckGUI` updates automatically when the user adjusts the speed slider. It calculates the visual readout using the formula `double currentBPM = baseBPM * speedRatio;`

iv. Supporting Screenshots



Figure 1

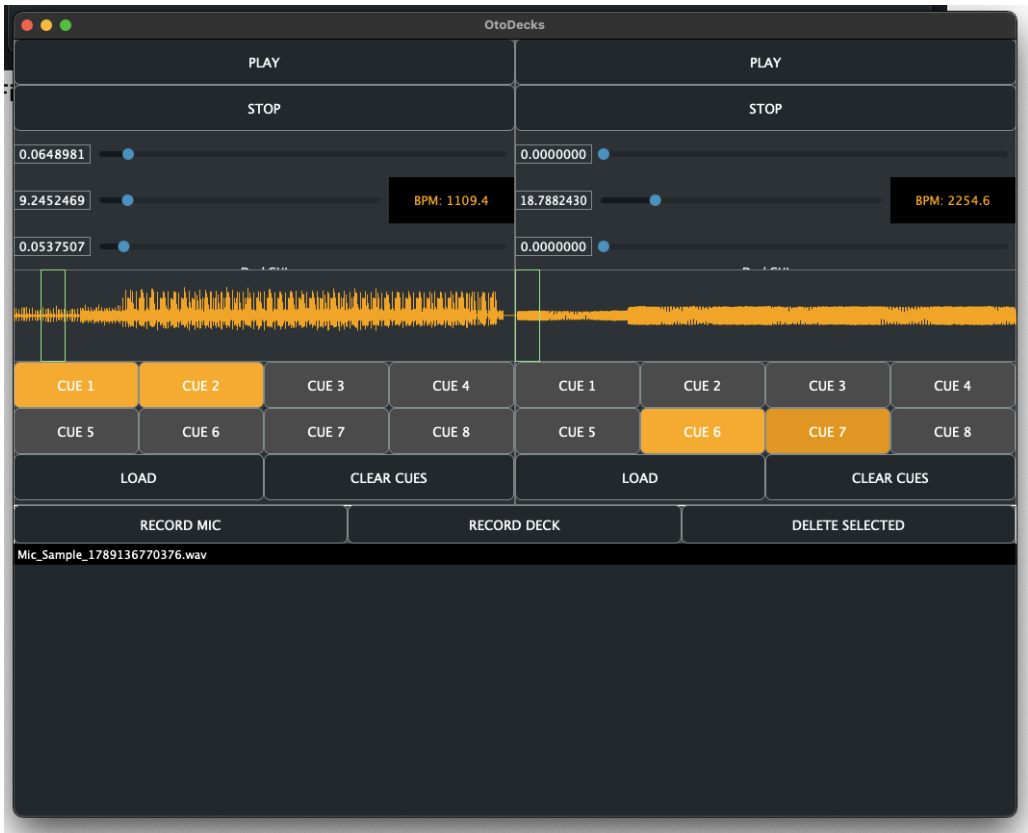


Figure 2

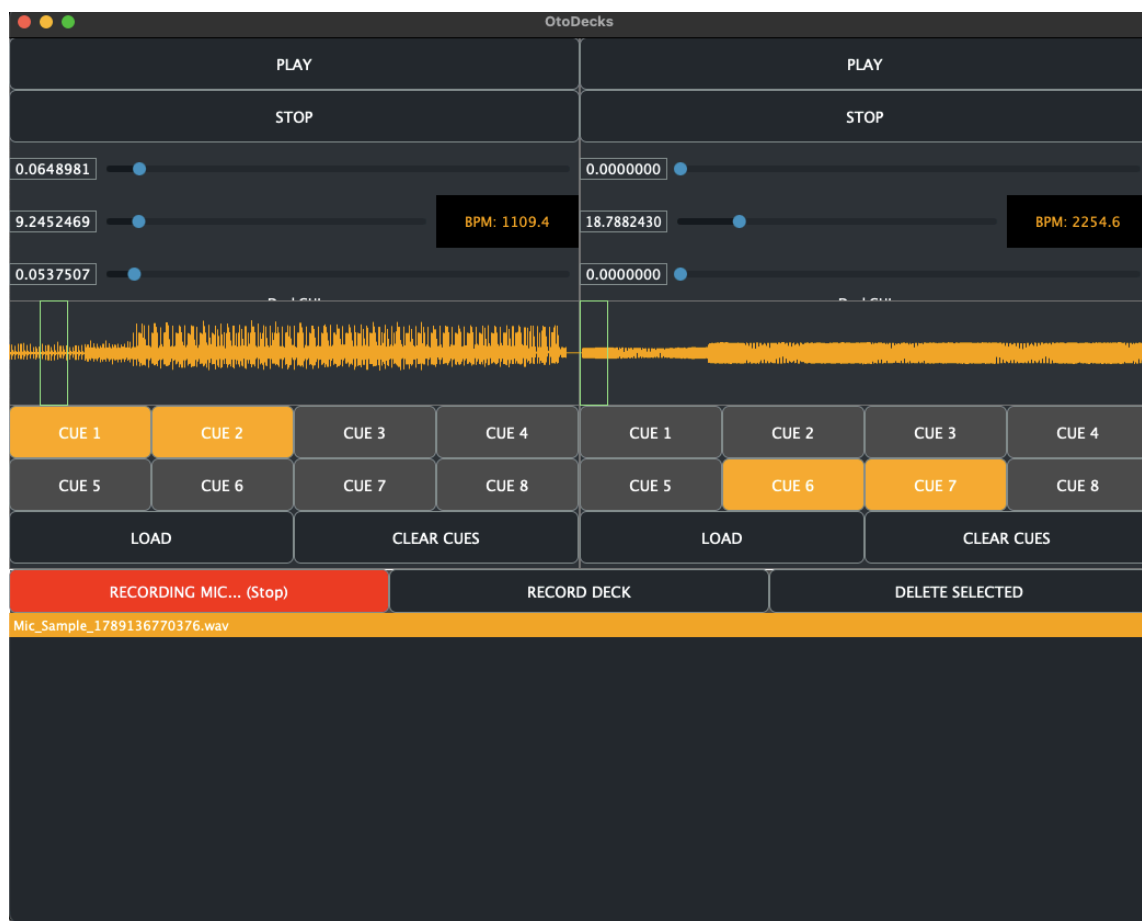


Figure 3